

PROVA TÉCNICA SUBITER

Respostas completas - Revisão 1

Soluções em C++/Qt e Flutter, com POO, SOLID, Clean Code, Clean Architecture, MVVM, Provider/ChangeNotifier, GoRouter, GetIt, i18n e SQLite.

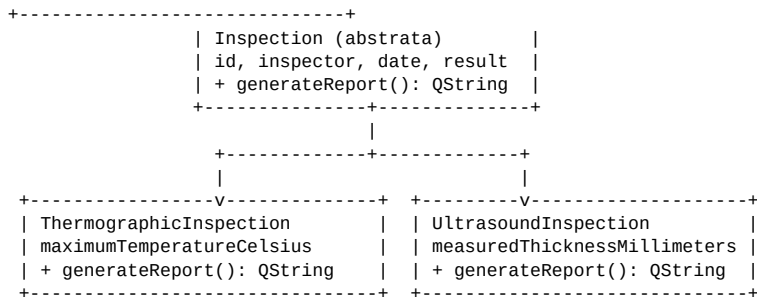
15 de julho de 2026

1. Gerenciamento de inspeções industriais em C++ e Qt

a) Modelagem da solução

A solução usa uma classe abstrata `Inspection` para os dados e o contrato comuns: ID, inspetor, data, resultado e geração de relatório. As classes `ThermographicInspection` e `UltrasoundInspection` especializam apenas o dado e a regra de relatório próprios de cada modalidade. Os consumidores trabalham com ponteiros para a abstração, permitindo polimorfismo em tempo de execução.

b) Diagrama de classes



Decisões de projeto

- Encapsulamento: atributos privados e dados comuns protegidos pela classe-base.
- SRP: cada modalidade conhece somente a geração de seu relatório.
- OCP/LSP: novos tipos podem implementar o contrato sem quebrar os consumidores.
- DIP: a aplicação depende de `Inspection`, e não das classes concretas.
- Qt: `QString`, `QDate`, `QCoreApplication` e `qInfo` integram a solução ao framework.

Trecho central

```

class Inspection {
public:
    virtual ~Inspection() = default;
    virtual QString generateReport() const = 0;
protected:
    QString commonReportData() const;
private:
    QString id_, inspector_, result_;
    QDate date_;
};

class ThermographicInspection final : public Inspection {
public:
    QString generateReport() const override;
private:
    double maximumTemperatureCelsius_;
};

class UltrasoundInspection final : public Inspection {
public:
    QString generateReport() const override;
private:
    double measuredThicknessMillimeters_;
};

```

A implementação C++/Qt completa, incluindo construtores, relatórios e exemplo de uso polimórfico, está em `subiter_test_c_qt/question1.h` e `subiter_test_c_qt/question1.cpp`. O executável usa o único ponto de entrada definido em `subiter_test_c_qt/main.cpp`.

2. Lista de inspeções consumindo API REST simulada

Comportamento entregue

O aplicativo lista inspeções termográficas e por ultrassom. O arquivo `assets/mocks/inspections.json` simula o retorno de uma API REST com `statusCode`, `message` e `data.inspections`. Cada card apresenta ID, tipo, equipamento, inspetor, data, local, resultado, resumo e medição.

Contrato JSON simulado

```
{
  "statusCode": 200,
  "message": "Inspeções carregadas com sucesso",
  "data": { "inspections": [ ... ] }
}
```

Arquitetura e fluxo

```
InspectionsScreen -> InspectionsViewModel (ChangeNotifier)
  -> GetInspectionsUseCase
  -> InspectionsRepository (contrato do domínio)
  -> InspectionsRepositoryImpl
      | -> JsonInspectionsRemoteDataSource -> JSON/REST simulado
      +--> SqliteInspectionsLocalDataSource -> SQLite
```

Separação entre interface e lógica

- UI: `InspectionsScreen` e `InspectionCard` apenas renderizam e encaminham ações.
- ViewModel: coordena o caso de uso e expõe estado e lista tipada.
- Domain: `InspectionModel`, enums, contrato do repositório e caso de uso.
- Infra: data model, parsing do JSON, SQLite e implementação do repositório.
- Injeção: `GetIt` cria clientes, fontes, repositórios, casos de uso e ViewModels.
- Navegação: rota nomeada `/inspections` integrada ao `GoRouter`.

Gerenciamento de estado

O ViewModel estende `ChangeNotifier` e é fornecido por `Provider`. Expõe `initial`, `loading`, `success`, `empty` e `error`. A tela usa `Consumer`, mostra progresso, lista, estado vazio ou erro e oferece `pull-to-refresh`.

Trecho do ViewModel

```
Future<void> getInspections() async {
  if (isLoading) return;
  _state = InspectionsViewState.loading;
  notifyListeners();
  try {
    final response = await _getInspectionsUseCase();
    _inspections = response.inspections;
    _state = _inspections.isEmpty
      ? InspectionsViewState.empty
      : InspectionsViewState.success;
  } on Object {
    _state = InspectionsViewState.error;
  }
  notifyListeners();
}
```

Tratamento de erros

A fonte valida status HTTP simulado, envelope, lista e campos obrigatórios. Payload inválido vira `DataException` na infraestrutura. Em falha, o repositório tenta o cache SQLite; sem cache, o ViewModel expõe `error` e a tela apresenta mensagem amigável e Tentar novamente. Uma API HTTP real pode substituir apenas o `RemoteDataSource`.

3. Cadastro de três equipamentos em C++

A classe `Equipment` representa nome, código, ID e descrição. A classe `EquipmentRegistry` tem responsabilidade exclusiva de armazenar no máximo três equipamentos e listá-los. A leitura exige texto não vazio, aceita espaços e valida ID inteiro positivo.

```
class Equipment {
public:
    Equipment(int id, std::string code,
              std::string name, std::string description);
    int id() const;
    const std::string& code() const;
    const std::string& name() const;
    const std::string& description() const;
private:
    int id_;
    std::string code_, name_, description_;
};

class EquipmentRegistry {
public:
    static constexpr std::size_t capacity = 3;
    void add(Equipment equipment);
    void list(std::ostream& output) const;
private:
    std::array<Equipment, capacity> equipments_;
    std::size_t size_ = 0;
};
```

Algoritmo

- Criar `EquipmentRegistry` com capacidade fixa igual a três.
- Repetir três vezes: ler ID, código, nome e descrição; validar; criar `Equipment`.
- Adicionar a entidade ao registro e impedir inserção além da capacidade.
- Ao final, percorrer os três objetos e imprimir todos os atributos.

Compilação

```
cd subiter_test_c_qt
cmake -S . -B build
cmake --build build
./build/SubiterInspections
```

O programa completo está em `subiter_test_c_qt/question3.h` e `subiter_test_c_qt/question3.cpp`. No Qt Creator, a opção Run in terminal deve estar habilitada porque a atividade usa entrada interativa com `std::cin`.

4. Cadastro de atividade em Flutter

A rota /activities/register apresenta os campos Nome da empresa e Local, ambos obrigatórios, e o botão Cadastrar. O formulário mantém TextEditingControllers na camada de apresentação e os descarta em dispose(). Após a validação, o ViewModel executa RegisterActivity.

Fluxo Clean Architecture/MVVM

```
ActivityRegistrationPage
-> ActivityRegistrationViewModel (ChangeNotifier)
-> RegisterActivity (valida e normaliza)
-> ActivitiesRepository (contrato)
-> ActivitiesRepositoryImpl
-> SQLite / tabela activities
```

O caso de uso remove espaços externos, cria a entidade Activity e persiste empresa, local e data do cadastro. O ViewModel expõe os estados initial, saving, success e failure. Durante a gravação, o botão é desabilitado para evitar duplicidade.

Trecho do caso de uso

```
Future<Activity> call({
  required String companyName,
  required String location,
}) {
  final company = companyName.trim();
  final place = location.trim();
  if (company.isEmpty || place.isEmpty) {
    throw const FormatException('Campos obrigatórios não informados.');
```

```
  }
  return _repository.save(Activity(
    companyName: company,
    location: place,
    createdAt: DateTime.now(),
  ));
}
```

Resultado exibido

Cadastro realizado com sucesso!
Empresa: ABC Engenharia - Local: São José dos Campos

A mensagem usa os valores realmente digitados. Textos de interface são obtidos pelo catálogo i18n, e o GoRouter/GetIt fazem navegação e injeção sem acoplar a página à infraestrutura.

5. Mapa da entrega e validação

Item	Local
Aplicação/rotas	lib/app/
i18n, banco e injeção	lib/core/
Questão 2	lib/modules/inspections/
JSON REST simulado	assets/mocks/inspections.json
Questão 4	lib/modules/activities/
Questões 1 e 3	subiter_test_c_qt/question1.* e question3.*
Testes	test/

Comandos de validação executados

```
flutter analyze
flutter test
cmake -S subiter_test_c_qt -B subiter_test_c_qt/build
cmake --build subiter_test_c_qt/build
```

- Análise estática Flutter: sem problemas.
- Testes automatizados: parsing, estados das inspeções, erros e cadastro validados.
- Questão 3: compilada com avisos rigorosos e executada com três registros.
- PDF final: renderizado e inspecionado página a página.

Execução do aplicativo

```
flutter pub get
flutter run
```

A arquitetura usa abstrações apenas onde há fronteiras reais (API, SQLite e casos de uso), evitando classes artificiais. Isso mantém SOLID e Clean Architecture de forma prática, legível e testável.